Žilinská univerzita v Žiline

Fakulta riadenia a informatiky

Algoritmy a údajové štruktúry 2

Semestrálna práca S1

Bc. Pavol Pohánka

2026/2027

Obsah

1 Architektúra aplikácie a návrh riešenia **1**

1.1 Projekt a jeho časti 1

1.2 Architektúra a tok údajov 1

2 Priestorové dátové štruktúry **3**

2.1 Spoločné rozhranie a body 3

2.2 KDTree 3

2.3 QuadTree 4

2.4 QuadKDTree 5

2.5 Porovnanie vlastných implementácií 6

3 Externé implementácie **7**

3.1 Externé KDTree 7

3.2 Externé QuadTree 7

4 Testovanie a porovnanie výkonnosti **8**

4.1 Testovací nástroj a dáta 8

4.2 Kontrola správnosti 8

4.3 Metodika benchmarku a obmedzenia 8

4.4 Výsledky testov 9

5 Aplikácia POI Manager **10**

5.1 Záznamy a priestorový index 10

5.2 Vyhľadávanie a správa POI 10

5.3 CSV a používateľské rozhranie 10

6 Používateľská príručka **11**

6.1 Spustenie 11

6.2 Každodenná práca 11

7 Záver **13**

Zoznam použitej literatúry **14**

Kapitola 1

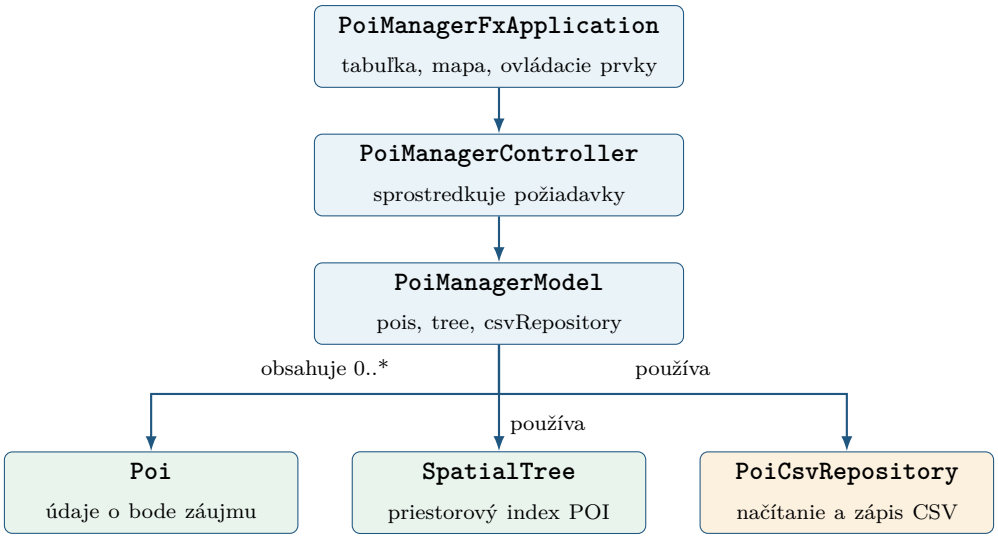
Architektúra aplikácie a návrh riešenia

1.1 Projekt a jeho časti

Projekt vznikol v jazyku Java 21 a zostavuje sa nástrojom Maven. Jeho základom sú tri vlastné priestorové stromy: KDTree, QuadTree a QuadKDTree. Na porovnanie je pripojených šesť externých implementácií. Súčasťou projektu je testovací nástroj a aplikácia POI Manager na správu bodov záujmu. Aplikácia má grafické rozhranie JavaFX a obsahuje aj terminálové rozhranie `ConsoleView`. Vstupným bodom je trieda `MainApplication`.

1.2 Architektúra a tok údajov

POI Manager tvorí používateľské rozhranie, riadiaca vrstva a model (obrázok 1.1). `PoiManager Controller` sprostredkúva požiadavky medzi rozhraním a `PoiManagerModel`. Model uchováva POI v zozname, na vyhľadávanie používa zvolený strom a prácu so súbormi CSV zabezpečuje cez `PoiCsvRepository`.



Obr. 1.1: UML diagram hlavných tried POI Managera

Grafické rozhranie vytvára `PoiManagerFxApplication`. Akcie v `PoiControlsView` a udalosti v tabuľke či na mape spúšťajú obslužné funkcie tejto triedy. Tie odovzdajú požiadavku triede `PoiManagerController`, ktorá zavolá príslušnú operáciu modelu. Výsledky sa následne zobrazia v `PoiTableView` a `PoiMapView`. Model nepracuje priamo s JavaFX; používateľské rozhranie tak možno meniť bez zásahu do priestorového vyhľadávania a práce s údajmi.

Balík `sk.fri.tester` obsahuje nástroj na overenie a porovnanie stromov, `sk.fri.adapters` obsahuje adaptéry externých stromov.

Kapitola 2

Priestorové dátové štruktúry

2.1 Spoločné rozhranie a body

Priestorový strom organizuje objekty podľa polohy. Pri vyhľadávaní v oblasti môže vynechať vetvy, ktoré neobsahujú hľadané body.

`SpatialTree<T,P>` je spoločná abstraktná trieda stromov, kde T je typ objektu a P typ bodu. Poskytuje operácie:

- `insert(T)` vloží objekt a vráti informáciu o úspechu,
- `find(P)` vyhľadá objekty na zadanej polohe,
- `rangeSearch(P min, P max)` vyhľadá objekty v oblasti určenej dolnými a hornými súradnicami.

Pri vytvorení stromu sa odovzdáva funkcia na získanie súradníc objektu a komparátor určujúci usporiadanie hodnôt súradníc.

Pre dvojrozmerné údaje sa používa `Point<K>` s dvojicou súradníc a pre k-rozmerný KDTree slúži `KDPoint<K>` so zoznamom súradníc. Pri hľadaní bodu sa musia zhodovať všetky jeho súradnice podľa komparátora.

Objekty s rovnakými súradnicami sa ukladajú do zoznamu hodnôt jedného uzla. Pri vyhľadávaní v oblasti sa započítajú aj body na jej okraji.

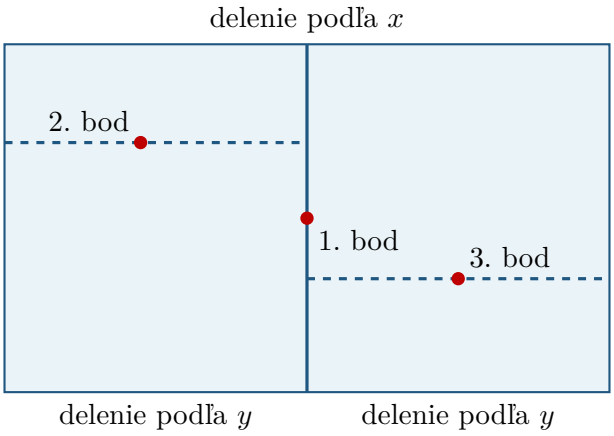
Vkladanie a hľadanie používajú cykly, pri prehľadávaní viacerých vetiev sa používa zásobník.

2.2 KDTree

`KDTree<T,K>` rozdeľuje priestor postupne podľa jednej súradnice. V dvojrozmernom prípade sa striedajú súradnice x a y, všeobecne sa používa dimenzia $d \bmod k$, kde d je hĺbka

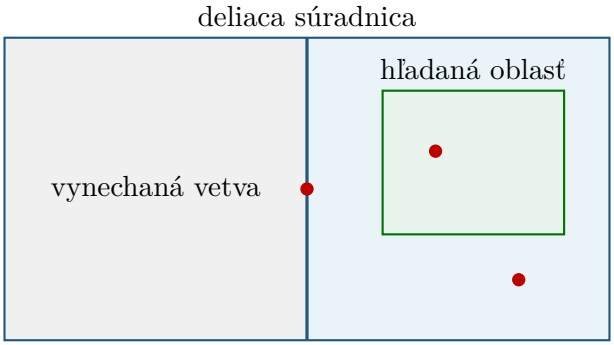
uzla. `KDNode` obsahuje bod, jeho hodnoty a odkazy na dvoch potomkov. Pri vkladaní a hľadaní bodu sa pokračuje vetvou podľa deliacej súradnice, pri jej zhode a rozdielnom bode sa pokračuje vpravo.

Príklad delenia je na obrázku 2.1.



Obr. 2.1: Príklad striedania súradníc pri delení v KDTree

Pri vyhľadávaní v oblasti strom kontroluje navštívené body a podľa hraníc oblasti vyberá vetvy, v ktorých sa môže nachádzať výsledok. Príklad je na obrázku 2.2.



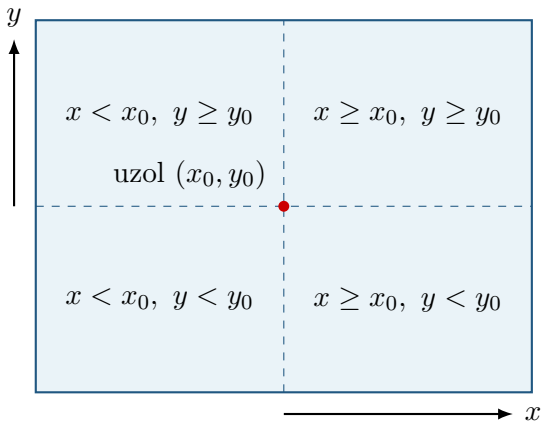
Obr. 2.2: Príklad vyhľadávania v oblasti ležiacej napravo od delenia

Pri výške stromu h trvá vkladanie a hľadanie bodu $O(hk)$. Bez vyvažovania môže byť h až n, počet rôznych bodov. Vyhľadávanie v oblasti trvá $O(vk + r)$, kde v je počet navštívených uzlov a r počet výsledkov, v najhoršom prípade $O(nk + r)$. Pre pevné k je pamäťová náročnosť lineárna vzhľadom na počet uzlov a uložených hodnôt.

2.3 QuadTree

`QuadTree<T,K>` rozdeľuje rovinu na štyri kvadranty podľa bodu uzla (obrázok 2.3). Uzol obsahuje bod, jeho hodnoty a až štyroch potomkov. Pri vkladaní aj hľadaní bodu sa podľa

súradníc vyberie príslušný kvadrant. Body na deliacej priamke patria napravo alebo nahor.



Obr. 2.3: Rozdelenie do kvadrantov podľa súradníc uzla QuadTree

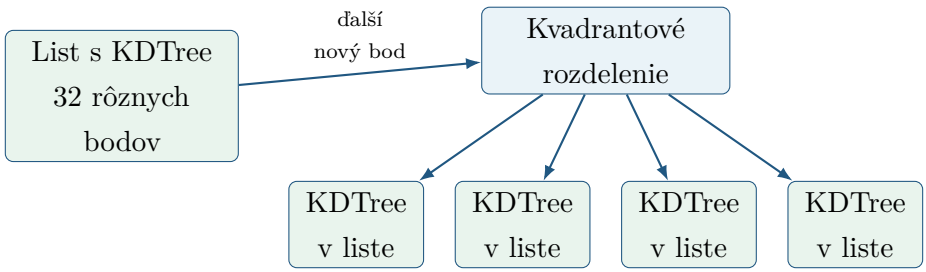
Pri vyhľadávaní v oblasti strom prechádza kvadranty, ktoré sa môžu prekrývať so zadanou oblasťou. Vkladanie a hľadanie bodu trvá $O(h)$, vyhľadávanie v oblasti $O(v+r)$, najviac $O(n+r)$. Pamäťová náročnosť je lineárna vzhľadom na počet uzlov a hodnôt.

Maximálna hĺbka je 32. Nový bod vyžadujúci hlbší uzol sa odmietne, k existujúcemu bodu možno pridať ďalšiu hodnotu.

2.4 QuadKDTree

`QuadKDTree<T,K>` kombinuje kvadrantové delenie s KDTree. Vonkajšia časť rozdeľuje priestor na kvadranty, zatiaľ čo body v každom liste sú uložené v samostatnom KDTree. Hodnoty s rovnakými súradnicami sa ukladajú spolu v uzle tohto KDTree.

List sa začne deliť pri vkladaní nového bodu, keď už obsahuje 32 bodov s rôznymi súradnicami a ešte nedosiahol limit hĺbky. Jeho hodnoty sa podľa polohy rozdelia do nových listov, ako ukazuje obrázok 2.4. Kvadrantové delenie má maximálnu hĺbku 32. Po dosiahnutí tohto limitu sa ďalšie body pridávajú do KDTree v liste, ktorý sa už ďalej nedelí.



Obr. 2.4: Príklad rozdelenia listu QuadKDTree do štyroch obsadených kvadrantov

Pri hľadaní bodu strom nájde príslušný list a prehľadá jeho KDTree. Pri vyhľadávaní v oblasti prehľadá KDTree v príslušných kvadrantoch. Pri výške vonkajšej cesty h a výške KDTree v liste h_k trvá hľadanie bodu aj vkladanie bez delenia $O(h + h_k k)$. Pri delení treba navyše rozmiestniť všetky hodnoty listu, vrátane hodnôt s rovnakými súradnicami. Vyhľadávanie v oblasti trvá v najhoršom prípade $O(nk + r)$. Pre pevné k je pamäťová náročnosť lineárna vzhľadom na počet uzlov a uložených hodnôt.

2.5 Porovnanie vlastných implementácií

Vlastnosť	**KDTree**	**QuadTree**	**QuadKDTree**
Rozdelenie priestoru	Striedanie súradnicových dimenzií	Kvadranty podľa bodu uzla	Kvadranty, KDTree v listoch
Dimenzie	k	2	2
Duplicitné body	Zoznam hodnôt v uzle	Zoznam hodnôt v uzle	Zoznam hodnôt v uzle KDTree
Vyhľadávanie v oblasti	Výber vetiev podľa deliacej súradnice	Výber kvadrantov podľa oblasti	Výber kvadrantov a vyhľadávanie v KDTree listov
Limity	–	Hĺbka 32, nové body nad limit sa odmietajú	Hĺbka 32, delenie listu pri 33. rôznom bode

Tabuľka 2.1: Vlastnosti vlastných priestorových štruktúr

Kapitola 3

Externé implementácie

Externé stromy umožňujú porovnať vlastné riešenie s ďalšími implementáciami. Každý z nich má adaptér odvodený od `SpatialTree`. Adaptér prevádza body a výsledky do formátu projektu, aby testovací nástroj mohol stromy používať rovnakým spôsobom.

3.1 Externé KDTree

- **KDtree-AlgorithmsAndStructures** [1] používa adaptér `KDAdapter`. Vkladanie a hľadanie bodu vykonáva strom, vyhľadávanie v oblasti adaptér rieši lineárnym prechodom cez pomocný zoznam bodov.
- **TinSpin Indexes** [2] sprístupňuje `TinSpinKDAdapter`. Súradnice prevádza na pole hodnôt `double` a vyhľadávanie vykonáva externý strom.
- **JTS** [3] používa `JtsKDAdapter`. Pre objekty s rovnakou polohou vedie pomocnú mapu hodnôt, oblasti zadáva pomocou triedy `Envelope`.

3.2 Externé QuadTree

- **Varun Pant Quadtree** [4] používa `VarunPantQuadTreeAdapter`, ktorý zoskupuje hodnoty na rovnakej polohe. Rozsah stromu je nastavený na zemepisné súradnice.
- **Alwex QuadTree** [5] sprístupňuje `AlwexQuadTreeAdapter`. Bod vkladá ako malý obdĺžnik a výsledné objekty filtruje podľa pôvodných súradníc. Implementácia používa súradnice typu `float`.
- **Java Simple Quadtree** [6] používa `RyanQuadTreeAdapter`. Body vkladá ako obdĺžniky nulovej veľkosti a výsledky filtruje podľa zadanej oblasti vrátane jej hraníc.

Kapitola 4

Testovanie a porovnanie výkonnosti

4.1 Testovací nástroj a dáta

`TreeTester` overuje tri vlastné stromy a šesť externých adaptérov. Vložené body vyhľadáva podľa polohy a výsledky hľadania v oblasti porovnáva s lineárnym prechodom prijatých bodov. Zo súborov `slovakia_nodes.csv` a `testovacie_oblasti.csv` načíta najviac 10 000 bodov a 1 000 oblastí.

4.2 Kontrola správnosti

Všetkých deväť konfigurácií prešlo kontrolami vyhľadávania. Pri kontrole vkladania a hľadania polohy sa vyberalo 1 000 bodov s opakovaním, pri hľadaní v oblasti sa použilo 10 000 bodov a 1 000 oblastí. Vlastný QuadTree odmietol 4 900 bodov pre limit hĺbky 32, preto sa výsledky porovnávali iba s prijatými bodmi.

4.3 Metodika benchmarku a obmedzenia

`TreeBenchmark` meria vkladanie, hľadanie vložených bodov a vyhľadávanie v oblasti. Výsledkom je medián časov v milisekundách. Každý beh vytvára nový strom, počet hľadaní sa môže líšiť podľa počtu úspešne vložených bodov.

Benchmark vykoná dva zahrievacie a päť meraných behov. Nástroj umožňuje použiť až 1 000 oblastí na beh, v uvedenom meraní bolo použitých 100.

4.4 Výsledky testov

Tabuľka uvádza mediány piatich behov s 10 000 vloženiami, 10 000 hľadaniami bodov a 100 dopytmi na oblasť na beh. Meranie sa uskutočnilo pred opravou počtu zahrievacích behov, po oprave sa benchmark znovu nespúšťal.

Implementácia	**Vkladanie**	**Hľadanie**	**Oblasť**	**Spolu**
My KDTree	5,266	5,056	0,512	10,833
My QuadTree	5,845	6,182	0,806	12,833
My Quad-KDTree	5,069	4,694	2,913	12,676
Existing external KDTree	6,242	6,264	21,801	34,307
TinSpin KDTree	4,254	25,984	0,446	30,683
JTS KDTree	11,958	13,796	0,325	26,078
Varun Pant QuadTree	4,171	4,316	0,069	8,557
Alwex QuadTree	18,314	128,441	5,067	151,822
Ryan Pelletier QuadTree	3,069	1 139,936	10,828	1 153,832

Tabuľka 4.1: Výsledky benchmarku: mediány v milisekundách

Najnižší celkový čas v uvedenom meraní má Varun Pant Quadtree (8,557 ms). Spomedzi vlastných štruktúr dosiahol najnižší celkový čas KDTree (10,833 ms).

Kapitola 5

Aplikácia POI Manager

5.1 Záznamy a priestorový index

`Poi` obsahuje identifikátor, názov, opis, súradnice, adresu a čas vytvorenia a poslednej úpravy. Model ukladá záznamy do zoznamu a ich polohu indexuje zvoleným stromom. Súradnice sú v poradí `(latitude, longitude)`.

5.2 Vyhľadávanie a správa POI

Vyhľadávanie podľa polohy vráti aktívne POI na zadaných súradniciach. Pri hľadaní v okolí sa najprv vyberú body v obdĺžniku a potom sa overí ich vzdialenosť. Pri hľadaní názvu v oblasti sa výsledky filtrujú podľa názvu bez ohľadu na veľkosť písmen a diakritiku.

POI možno vytvárať, upravovať, zobrazovať a mazať. Mazanie je logické: záznam zostáva v pamäti, ale bežné vyhľadávanie ho vynechá. Pri úprave sa index zostaví znova. Skúšobné POI sa generujú v obdĺžnikovej oblasti, preto môžu ležať aj mimo hranice Slovenska.

5.3 CSV a používateľské rozhranie

CSV súbor obsahuje iba aktívne záznamy. Polia sú oddelené bodkočiarkou, preto názov, opis ani adresa nesmú obsahovať bodkočiarku alebo nový riadok. Zmeny sa ukladajú akciou používateľa.

Rozhranie JavaFX zobrazuje POI v tabuľke a na offline mape Slovenska [7]. Mapa umožňuje posun, zmenu mierky, výber bodu a vytvorenie POI kliknutím.

Kapitola 6

Používateľská príručka

6.1 Spustenie

Na spustenie je potrebný JDK 21 a Maven. V koreňovom adresári projektu spustite:

```
mvn javafx:run
```

K dispozícii je aj spustiteľná verzia aplikácie pre Windows vo formáte `.exe`. Po spustení vyberte v úvodnom dialógu KD Tree, Quad Tree alebo Quad KD Tree. V nasledujúcom kroku zvoľte „New file“ a určte nový CSV súbor, alebo vyberte „Open file“ a načítajte existujúci CSV súbor s bodmi záujmu. Zvolený strom zostane aktívny počas celého behu aplikácie, zmeniť ho možno až po jej opätovnom spustení.

Projekt obsahuje aj terminálové rozhranie `ConsoleView`, jeho spustenie je v `Main Application` zakomentované.

6.2 Každodenná práca

1. Na hľadanie podľa polohy zvoľte „Find by GPS“ a zadajte súradnice v desatinných stupňoch.
2. Na vyhľadanie okolia kliknite na riadok POI v tabuľke, použite „Find nearby“ a zadajte polomer v metroch. Stredom je poloha vybraného záznamu.
3. Na hľadanie podľa názvu použite „Search by name“ a zadajte text aj hranice oblasti.
4. Na vytvorenie POI použite „Add POI“, prípadne kliknite do mapy vnútri hranice Slovenska. Na úpravu zaškrtnite práve jeden aktívny záznam a zvoľte „Edit selected“.
5. Na vymazanie zaškrtnite záznamy a použite „Delete selected“. Cez „Show deleted“ možno zobraziť aj vymazané záznamy.

6. Rozsah zobrazených záznamov sa mení v ponuke „Show“, „All POI“ zobrazí všetky. „Generate test POIs“ vytvorí skúšobné záznamy.

7. Pred zatvorením alebo prepnutím súboru uložte zmeny cez „Save CSV“, automatické ukladanie nie je zapnuté.

Kapitola 7

Záver

Výsledkom práce sú tri vlastné priestorové štruktúry, šesť externých adaptérov a aplikácia POI Manager, ktorá index využíva na hľadanie POI podľa polohy, oblasti a vzdialenosti. V zaznamenanom teste správnosti deväť konfigurácií prešlo kontrolami nad skúšobnou vzorkou, vlastný QuadTree však pre limit hĺbky odmietol časť bodov. Benchmarkové výsledky zaznamenávajú časy namerané pri použitom nastavení testu.

Zoznam použitej literatúry

[1] Luka3691. *KDtree-AlgorithmsAndStructures*. `https://github.com/luka3691/KDtree-AlgorithmsAndStructures`. [cit. 2026-11-09].

[2] Tzaeschke. *TinSpin Indexes*. `https://github.com/tzaeschke/tinspin-indexes`. [cit. 2026-11-09].

[3] LocationTech. *JTS Topology Suite*. `https://github.com/locationtech/jts`. [cit. 2026-11-09].

[4] Varun Pant. *Quadtree*. `https://github.com/varunpant/Quadtree`. [cit. 2026-11-09].

[5] Alwex. *QuadTree*. `https://github.com/alwex/QuadTree`. [cit. 2026-11-09].

[6] Ryan Pelletier. *Java Simple Quadtree*. `https://github.com/ryanmpelletier/java-simple-quadtree`. [cit. 2026-11-09].

[7] Natural Earth. *Natural Earth – public domain map data*. `https://www.naturalearthdata.com/`. [cit. 2026-11-09]. Zdrojová poznámka je uvedená v `src/main/resources/map/slovakia-border.txt`.